Hato: guía de diseño del backend (Java 21, POO)

Versión 1.0 · Octubre 2026 · Documento de trabajo para el desarrollo del servidor

1. Qué hace el backend

El backend es el cerebro de Hato. Los GPS y la app Android nunca hablan entre sí: todo pasa por el servidor, que debe:

1. **Recibir** las posiciones de los GPS (por TCP o desde Traccar).
2. **Guardarlas** y entregarlas en tiempo real a la app.
3. **Evaluar reglas**: salida de zona, sin señal, batería baja, quieto demasiado tiempo, velocidad sospechosa.
4. **Avisar** por notificación push (FCM) y canales de respaldo, aunque la app esté cerrada.
5. **Servir** la API que usa la app: usuarios, fincas, animales, zonas, alertas, historial y reportes.

2. Tecnologías

Pieza	Elección	Motivo
Lenguaje	Java 21 (LTS)	Records, sealed interfaces, pattern matching en switch e hilos virtuales
Framework	Spring Boot 3.2 o superior	Soporte oficial de Java 21 y de hilos virtuales (verificar la versión vigente)
Base de datos	PostgreSQL + PostGIS	Consultas geográficas y de historial
Migraciones	Flyway	El esquema queda versionado
Tiempo real	WebSocket de Spring	Empuja posiciones y alertas a la app abierta
Avisos	Firebase Admin SDK (FCM) + API de Telegram	Push como canal principal y respaldo externo

Pieza	Elección	Motivo
Geometría	Clases propias de dominio, con JTS opcional en la capa de infraestructura	El dominio no depende de una librería
Pruebas	JUnit 5, Mockito, Testcontainers	Pruebas unitarias y de integración reales
Construcción	Maven o Gradle	A elección del equipo

3. Principios de programación orientada a objetos

1. **Una clase, una responsabilidad.** Cada clase hace una sola cosa: decodificar, evaluar una regla, enviar un aviso.
2. **Programar contra interfaces.** Lo que puede cambiar (el modelo de GPS, el canal de avisos, el tipo de zona) se esconde detrás de una interfaz. Así se agrega un GPS nuevo sin tocar el resto.
3. **Datos inmutables.** Los datos que viajan (posiciones, eventos, DTOs) son `record`, que no se pueden modificar después de creados.
4. **Estados cerrados con `sealed`.** Los tipos de evento y de zona son jerarquías selladas: el compilador obliga a cubrir todos los casos.
5. **Polimorfismo en lugar de `if` gigantes.** Cada regla de alerta es una clase; cada protocolo GPS, un decodificador; cada canal, un notificador.
6. **Inyección de dependencias por constructor.** Las clases reciben lo que necesitan y se prueban sin Spring.
7. **Encapsulación.** Las entidades validan sus propios datos y no exponen sus colecciones internas.

Patrones que se usan: Strategy (reglas, decodificadores, notificadores), Factory (decodificadores), Observer (eventos de dominio), Repository (acceso a datos) y Facade (los servicios de aplicación).

4. Arquitectura por capas



Cuatro capas, con dependencias solo hacia adentro:

- **domain:** entidades, records, interfaces y reglas de negocio. No conoce Spring ni la base de datos.
- **application:** servicios que orquestan casos de uso (`PositionService` , `AlertService`).
- **infrastructure:** implementaciones concretas (JPA, TCP, FCM, WebSocket).
- **api:** controladores REST, DTOs y seguridad.

5. Estructura del proyecto

```
co.hato
 ├─ domain
 |   ├─ user         User, Role, Membership
 |   ├─ farm         Farm
 |   ├─ animal       Animal, Tracker, TrackerStatus
 |   ├─ position     GeoPoint, Position
 |   ├─ geofence     Geofence, PolygonGeofence, CircleGeofence, ZoneSchedule
 |   ├─ alert        Alert, AlertEvent, AlertRule + reglas, Severity
 |   └─ notification Notifier, NotificationMessage, Recipient
 ├─ application    PositionService, ZoneService, AlertService, RuleEngine,
 |                 AnimalService, FarmService, AuthService, ReportService
 ├─ infrastructure
 |   ├─ gps          GpsReceiver, TcpGpsReceiver, ProtocolDecoder,
Gt06Decoder, DecoderFactory
 |   ├─ persistence Repositorios JPA y mapeadores
 |   ├─ messaging   FcmNotifier, TelegramNotifier, NotificationDispatcher
 |   ├─ realtime    LivePositionPublisher, LiveSocketHandler
 |   └─ scheduling  SignalWatchdog, BatteryMonitor, IdleDetector
 └─ api            Controllers, DTOs, JwtFilter, GlobalExceptionHandler
```

6. Clases principales por módulo

Módulo	Clase	Responsabilidad
Usuarios	`User`, `Role` (enum), `Membership`	Persona, su rol (Propietario, Capataz, Solo lectura) y acceso a cada finca
Usuarios	`AuthService`	Registro, ingreso y emisión de tokens JWT
Fincas	`Farm`, `FarmService`	Finca y sus miembros
Animales	`Animal`, `AnimalService`	Ficha del animal y su vínculo con un GPS
Dispositivos	`Tracker`, `TrackerStatus`	GPS físico: identificador, batería, última conexión
Posiciones	`GeoPoint`, `Position`	Coordenadas y lectura del GPS (records inmutables)
Ingesta	`GpsReceiver` (interfaz), `TcpGpsReceiver`	Escucha conexiones de los GPS

Módulo	Clase	Responsabilidad
Ingesta	`ProtocolDecoder` (interfaz), `Gt06Decoder`, `H02Decoder`, `DecoderFactory`	Traducen los bytes de cada marca de GPS a `Position`
Ingesta	`PositionService`, `PositionRepository`	Guardar, publicar y disparar la evaluación de reglas
Zonas	`Geofence` (abstracta), `PolygonGeofence`, `CircleGeofence`	Saber si un punto está dentro y a cuántos metros del límite
Zonas	`ZoneSchedule`, `ZoneService`	Horarios de vigilancia y administración de zonas
Alertas	`AlertEvent` (sealed), `Alert`, `Severity`	Lo que ocurrió y su gravedad
Alertas	`AlertRule` (interfaz) y una clase por regla	Cada regla decide si hay alerta
Alertas	`RuleEngine`, `AlertService`, `AlertDeduplicator`	Ejecutar reglas, guardar alertas y evitar repetidas
Avisos	`Notifier` (interfaz), `FcmNotifier`, `TelegramNotifier`	Un canal de envío por clase
Avisos	`NotificationDispatcher`, `NotificationTemplate`	Elegir canales y redactar el mensaje
Tiempo real	`LivePositionPublisher`	Empujar posiciones y alertas por WebSocket
Tareas	`SignalWatchdog`, `BatteryMonitor`, `IdleDetector`	Detectar lo que **no** llega: silencio, batería, inmovilidad
Reportes	`TrackService`, `ReportService`, `PdfReportGenerator`	Recorridos, resúmenes y PDF

7. Código de las clases clave (Java 21)

7.1 Datos inmutables: records

```
public record GeoPoint(double lat, double lng) {
    public GeoPoint {
        if (lat < -90 || lat > 90 || lng < -180 || lng > 180)
            throw new IllegalArgumentException("Coordenadas fuera de rango");
```

```
    }

    /** Distancia en metros (fórmula de Haversine). */
    public double distanceMetersTo(GeoPoint o) {
        double r = 6_371_000;
        double dLat = Math.toRadians(o.lat - lat);
        double dLng = Math.toRadians(o.lng - lng);
        double a = Math.pow(Math.sin(dLat / 2), 2)
                 + Math.cos(Math.toRadians(lat)) *
Math.cos(Math.toRadians(o.lat))
                 * Math.pow(Math.sin(dLng / 2), 2);
        return 2 * r * Math.asin(Math.sqrt(a));
    }
}

public record Position(UUID trackerId, GeoPoint point, Instant recordedAt,
                       double speedKmh, int batteryPercent, double
accuracyMeters) {}
```

7.2 Zonas: clase abstracta con polimorfismo

```
public abstract sealed class Geofence permits PolygonGeofence, CircleGeofence
{
    private final UUID id;
    private final String name;
    private final double toleranceMeters;

    protected Geofence(UUID id, String name, double toleranceMeters) {
        this.id = id; this.name = name; this.toleranceMeters =
toleranceMeters;
    }

    public abstract boolean contains(GeoPoint p);
    public abstract double distanceToBoundaryMeters(GeoPoint p);

    /** Fuera de verdad: salió y superó el margen de tolerancia del GPS. */
    public boolean isOutside(GeoPoint p) {
        return !contains(p) && distanceToBoundaryMeters(p) > toleranceMeters;
    }

    public UUID id() { return id; }
    public String name() { return name; }
}

public final class CircleGeofence extends Geofence {
    private final GeoPoint center;
```

```
    private final double radiusMeters;

    public CircleGeofence(UUID id, String name, double tolerance, GeoPoint
center, double radius) {
        super(id, name, tolerance);
        this.center = center; this.radiusMeters = radius;
    }

    @Override public boolean contains(GeoPoint p) {
        return center.distanceMetersTo(p) <= radiusMeters;
    }
    @Override public double distanceToBoundaryMeters(GeoPoint p) {
        return Math.abs(center.distanceMetersTo(p) - radiusMeters);
    }
}
```

`PolygonGeofence` guarda una lista de `GeoPoint` (mínimo 3, validado en el constructor) y usa el algoritmo del rayo para `contains`. La distancia al límite se calcula con la distancia mínima del punto a cada lado, en una clase utilitaria `GeometryUtils`.

7.3 Eventos y reglas de alerta

```
public sealed interface AlertEvent {
    UUID animalId();
    Instant at();
    Severity severity();

    record ExitedZone(UUID animalId, UUID zoneId, Instant at, double
metersOut) implements AlertEvent {
        public Severity severity() { return Severity.HIGH; }
    }
    record ReturnedToZone(UUID animalId, UUID zoneId, Instant at) implements
AlertEvent {
        public Severity severity() { return Severity.LOW; }
    }
    record LowBattery(UUID animalId, Instant at, int percent) implements
AlertEvent {
        public Severity severity() { return Severity.MEDIUM; }
    }
    record NoSignal(UUID animalId, Instant at, Duration silentFor) implements
AlertEvent {
        public Severity severity() { return Severity.MEDIUM; }
    }
    record Idle(UUID animalId, Instant at, Duration idleFor) implements
AlertEvent {
        public Severity severity() { return Severity.HIGH; }
```

```
    }
}

public record AlertContext(Animal animal, Position current,
                           Optional<Position> previous, List<Geofence> zones)
{}

public interface AlertRule {
    List<AlertEvent> evaluate(AlertContext ctx);
}

@Component
public class OutOfZoneRule implements AlertRule {
    @Override
    public List<AlertEvent> evaluate(AlertContext ctx) {
        GeoPoint now = ctx.current().point();
        return ctx.zones().stream()
            .filter(z -> z.isOutside(now))
            .filter(z -> ctx.previous().map(p ->
!z.isOutside(p.point())).orElse(true))
            .<AlertEvent>map(z -> new
AlertEvent.ExitedZone(ctx.animal().id(), z.id(),
                    ctx.current().recordedAt(),
z.distanceToBoundaryMeters(now)))
            .toList();
    }
}
```

La regla solo dispara en la **transición** de dentro a fuera, no con cada posición recibida. Así se evita el spam de notificaciones. `RuleEngine` recibe `List<AlertRule>` por inyección: agregar una regla nueva es crear una clase con `@Component`.

7.4 Avisos: un canal por clase

```
public interface Notifier {
    boolean supports(Severity severity);
    void send(Recipient to, NotificationMessage message);
}

@Service
public class NotificationDispatcher {
    private final List<Notifier> notifiers;
    private final ExecutorService pool =
Executors.newVirtualThreadPerTaskExecutor();

    public NotificationDispatcher(List<Notifier> notifiers) { this.notifiers
```

```
= notifiers; }

    public void dispatch(AlertEvent event, List<Recipient> recipients) {
        NotificationMessage msg = NotificationTemplate.from(event);
        for (Recipient r : recipients)
            for (Notifier n : notifiers)
                if (n.supports(event.severity()))
                    pool.submit(() -> n.send(r, msg));
    }
}

public final class NotificationTemplate {
    public static NotificationMessage from(AlertEvent event) {
        return switch (event) {
            case AlertEvent.ExitedZone e -> new NotificationMessage("Un
animal salió de la zona",
                    "A %.0f m del límite".formatted(e.metersOut()),
Severity.HIGH);
            case AlertEvent.ReturnedToZone e -> new NotificationMessage("Un
animal volvió", "Ya está dentro de la zona", Severity.LOW);
            case AlertEvent.LowBattery e -> new NotificationMessage("Batería
baja", "Queda el " + e.percent() + " %", Severity.MEDIUM);
            case AlertEvent.NoSignal e -> new NotificationMessage("Sin
señal", "Silencio de " + e.silentFor().toHours() + " h", Severity.MEDIUM);
            case AlertEvent.Idle e -> new NotificationMessage("Animal
quieto", "Lleva " + e.idleFor().toHours() + " h sin moverse", Severity.HIGH);
        };
    }
}
```

El `switch` no necesita `default`: como `AlertEvent` es sellada, si se agrega un evento nuevo el compilador marca todos los lugares que faltan por cubrir.

7.5 Ingesta de posiciones

```
@Service
public class PositionService {
    private final PositionRepository positions;
    private final AnimalService animals;
    private final ZoneService zones;
    private final RuleEngine rules;
    private final AlertService alerts;
    private final LivePositionPublisher live;
    // constructor con todas las dependencias

    @Transactional
```

```
    public void ingest(Position p) {
        if (positions.exists(p.trackerId(), p.recordedAt())) return; //
idempotente
        Optional<Position> previous = positions.findLatest(p.trackerId());
        positions.save(p);
        live.publish(p);

        animals.findByTracker(p.trackerId()).ifPresent(animal -> {
            var ctx = new AlertContext(animal, p, previous,
zones.activeFor(animal));
            alerts.register(rules.evaluate(ctx));
        });
    }
}
```

Importante: con la memoria offline de los GPS, las posiciones pueden llegar **desordenadas o tarde**. Por eso se ordena y se compara por `recordedAt` (hora del GPS), no por la hora de llegada, y solo se evalúan las reglas con la posición más reciente.

7.6 Receptor TCP con hilos virtuales

```
public class TcpGpsReceiver implements GpsReceiver {
    private final int port;
    private final DecoderFactory decoders;
    private final PositionService positions;
    private volatile boolean running = true;

    @Override
    public void start() throws IOException {
        try (var server = new ServerSocket(port);
             var pool = Executors.newVirtualThreadPerTaskExecutor()) {
            while (running) {
                Socket socket = server.accept();
                pool.submit(() -> handle(socket)); // un hilo virtual por GPS
conectado
            }
        }
    }
}
```

Los hilos virtuales permiten miles de conexiones de GPS abiertas a la vez con código simple y bloqueante, sin programación reactiva. `DecoderFactory` recibe todos los `ProtocolDecoder` y escoge el que reconoce los primeros bytes de la conexión.

Atajo recomendado al comienzo: usar Traccar como receptor y que reenvíe cada posición a `POST /api/ingest/positions`. Así no hay que escribir decodificadores propios hasta que realmente hagan falta.

8. Modelo de datos

Tabla	Columnas principales
`users`	id, nombre, correo, teléfono, hash de clave
`farms`	id, nombre, ubicación
`memberships`	user_id, farm_id, rol
`animals`	id, farm_id, nombre, arete, raza, edad, foto, notas, tracker_id
`trackers`	id, código (IMEI), batería, última conexión
`positions`	tracker_id, recorded_at, ubicación `geography(Point,4326)`, velocidad, batería, precisión
`zones`	id, farm_id, nombre, tipo (POLYGON o CIRCLE), geometría, radio, tolerancia, horario, activa
`zone_animals`	zone_id, animal_id
`alerts`	id, animal_id, tipo, severidad, datos (JSON), creada, atendida_por, atendida_en
`device_tokens`	user_id, token FCM, plataforma
`notification_prefs`	user_id, tipo de alerta, canal, activo

Notas: clave única `(tracker_id, recorded_at)` para que reenviar una posición no la duplique, índice GiST sobre la ubicación, tabla `positions` particionada por mes y una política de retención (por ejemplo, 12 meses) para que no crezca sin límite.

9. Reglas de alerta

Regla	Condición	Severidad	Enfriamiento
`OutOfZoneRule`	Pasa de dentro a fuera, más allá de la tolerancia	Alta	Hasta que regrese

Regla	Condición	Severidad	Enfriamiento
`BackInZoneRule`	Pasa de fuera a dentro	Baja	Ninguno
`LowBatteryRule`	Batería menor a 20 %	Media	6 horas
`NoSignalRule`	Sin posiciones por más de X horas (la ejecuta `SignalWatchdog`)	Media	6 horas
`IdleTooLongRule`	Sin desplazarse más de 20 m en X horas	Alta	12 horas
`SpeedRule`	Velocidad por encima de lo normal (posible vehículo)	Alta	30 minutos
`MassExitRule`	Varios animales salen de la misma zona en pocos minutos	Crítica	30 minutos

`AlertDeduplicator` guarda el último estado por animal y zona para que no se repita una alerta ya enviada.

10. API REST y WebSocket

Método y ruta	Función
`POST /api/auth/login`, `POST /api/auth/register`	Ingreso y registro
`GET /api/farms`	Fincas del usuario
`GET /api/farms/{id}/animals`	Animales con su última posición y estado
`POST /api/animals`, `PUT /api/animals/{id}`	Crear y editar animales
`GET /api/animals/{id}/track?from=&to=`	Recorrido por rango de fechas
`GET/POST/PUT/DELETE /api/zones`	Administrar zonas
`GET /api/alerts?status=pending`	Alertas pendientes o historial
`POST /api/alerts/{id}/ack`	Marcar como atendida
`POST /api/devices/token`	Registrar el token FCM del celular
`GET /api/reports/exits?from=&to=`	Reporte (PDF o JSON)
`POST /api/ingest/positions`	Entrada para Traccar (protegida con clave de servicio)

Método y ruta	Función
`WS /ws/live?farmId=`	Posiciones y alertas en tiempo real

Seguridad: HTTPS en todo, JWT con vencimiento corto, verificación de rol y de pertenencia a la finca en cada ruta, límite de peticiones por usuario y validación de todos los DTO (también `record`).

11. Tareas programadas

- **`SignalWatchdog` (cada minuto):** busca GPS que llevan más del tiempo configurado sin reportar y genera `NoSignal`. Es necesaria porque un GPS sin señal justamente **no envía nada** que dispare una regla.
- **`BatteryMonitor`:** revisa la batería de cada GPS.
- **`IdleDetector`:** compara las últimas posiciones para detectar inmovilidad.
- **Limpieza de datos:** borra o archiva posiciones antiguas.

12. Pruebas

- **Unitarias (sin Spring):** `GeoPoint`, `PolygonGeofence`, cada `AlertRule` y `NotificationTemplate`. Al ser clases puras, se prueban en milisegundos.
- **Integración:** repositorios y consultas geográficas con Testcontainers y una imagen de PostGIS.
- **De extremo a extremo:** un simulador de GPS (como el que ya probamos en el prototipo) que envía una ruta que sale de la zona y comprueba que se genera una sola alerta.
- **Casos especiales a cubrir:** posiciones desordenadas, posiciones duplicadas, GPS con precisión mala, zona con horario y salida de varios animales a la vez.

13. Despliegue

- Imagen Docker basada en `eclipse-temurin:21-jre`, con PostgreSQL/PostGIS en un contenedor aparte (por ejemplo, con docker-compose).
- Variables de entorno para claves: base de datos, JWT, credenciales de Firebase y token de Telegram. Nunca dentro del código.
- **Puerto TCP abierto para los GPS** (por ejemplo, el 5023). Muchos servicios gratuitos de hosting solo permiten tráfico HTTP, así que para recibir GPS directamente conviene un VPS económico. Si se usa Traccar desde el comienzo, este punto se simplifica.

- Copias de seguridad diarias de la base de datos y monitoreo de salud (`/actuator/health`).

14. Orden sugerido de desarrollo

1. **Núcleo del dominio:** `GeoPoint`, `Position`, `Geofence` y sus pruebas unitarias.
2. **Persistencia:** esquema con Flyway y repositorios.
3. **Ingesta:** `POST /api/ingest/positions` y `PositionService`, probado con el simulador.
4. **Alertas:** `RuleEngine`, `OutOfZoneRule` y `AlertService`.
5. **Avisos:** `FcmNotifier` y registro de tokens desde la app.
6. **API para la app:** autenticación, animales, zonas y WebSocket.
7. **Reglas restantes y tareas programadas:** batería, sin señal, quieto, velocidad.
8. **Receptor TCP propio y decodificadores**, o integración definitiva con Traccar.
9. **Reportes, roles y varias fincas.**

15. El backend de un vistazo

Esta parte reúne en un solo lugar lo que se decidió a lo largo del proyecto: lo que se probó en el prototipo, lo que exigen los GPS reales y lo que la app necesita del servidor.

Etapa	Qué ocurre	Clases responsables
Entrada	El GPS (o Traccar, o una antena LoRa) envía una posición	`TcpGpsReceiver`, `PositionIngestController`, `MqttGpsReceiver`
Normalización	Se convierte a un `Position` único, sin importar la marca del GPS	`ProtocolDecoder`, `DecoderFactory`
Guardado	Se ignoran duplicados y se almacena	`PositionService`, `PositionRepository`
Tiempo real	Se empuja a las apps abiertas	`LivePositionPublisher`
Evaluación	Se aplican las reglas sobre la posición	`RuleEngine`, `AlertRule` y sus reglas
Control de repetidos	Una alerta por evento, no por posición	`AlertDeduplicator`, `ExitConfirmationPolicy`

Etapa	Qué ocurre	Clases responsables
Aviso	Push, Telegram u otros canales	`NotificationDispatcher`, `Notifier`
Vigilancia de lo que no llega	Silencio, batería, inmovilidad	`SignalWatchdog`, `BatteryMonitor`, `IdleDetector`
Consulta	La app pide mapa, historial y reportes	Controladores REST, `TrackService`, `ReportService`

Decisiones tomadas hasta ahora:

- **Java 21 con Spring Boot**, organizado en capas y con una clase por responsabilidad.
- **PostgreSQL con PostGIS** para posiciones, zonas y alertas.
- **La detección de salida de zona se hace en el servidor**, nunca en la app, porque debe funcionar con la app cerrada.
- **Traccar como primera opción** para recibir los GPS, con receptor propio solo si hace falta.
- **Notificaciones push (FCM)** como canal principal y Telegram como respaldo.

16. Conexión con los GPS reales

16.1 Tres formas de entrada, una sola interfaz

Todas implementan `GpsReceiver` y entregan el mismo `Position` al servicio. Cambiar de una a otra no toca las reglas ni la app.

Opción	Cómo funciona	Cuándo conviene
A. Traccar + reenvío	Traccar recibe el GPS y reenvía cada posición a `POST /api/ingest/positions`	Para empezar: soporta más de 200 modelos sin escribir decodificadores
B. Receptor TCP propio	`TcpGpsReceiver` con un `ProtocolDecoder` por marca	Cuando se fija un modelo de GPS y se quiere menos piezas
C. LoRa	Collar → antena base (gateway) → servidor LoRa → MQTT o HTTP → `MqttGpsReceiver`	Fincas sin cobertura celular

16.2 Lo que debe cumplir el GPS que se compre

- Protocolo conocido y compatible con Traccar (por ejemplo GT06 o H02).
- Red 4G (las redes 2G se están apagando) o LoRa si no hay cobertura celular.
- **Memoria sin conexión:** guarda posiciones cuando no hay señal y las envía en bloque al volver.
- Intervalo de reporte configurable y batería de larga duración.
- Hora del GPS en cada posición (el backend usa `recordedAt` y no la hora de llegada).

16.3 Poca señal: cómo lo maneja el servidor

En una finca con cobertura irregular las posiciones llegan tarde, juntas y desordenadas. El backend lo soporta así:

1. **Idempotencia:** la clave única `(tracker_id, recorded_at)` evita duplicados si el GPS reenvía.
2. **Orden por hora del GPS:** una posición vieja que llega tarde se guarda en el historial, pero **no dispara alertas** si ya existe una más reciente.
3. **Umbral de «sin señal» dependiente del GPS:** cada `Tracker` guarda su `reportIntervalSeconds`, y `NoSignalRule` alerta cuando pasa más de `noSignalFactor × intervalo` (por defecto 3 veces), no un número fijo para todos.
4. **Estado en la app:** mientras no llegan datos, el mapa muestra la última posición con la hora.

16.4 Menos falsas alarmas

El GPS tiene un error normal de 5 a 15 metros, y a veces mucho más. Dos protecciones:

- **Filtro de precisión:** las posiciones con `accuracy` peor que el máximo configurado (por defecto 50 m) se guardan, pero no se usan para decidir si salió de la zona.
- **Confirmación:** `ExitConfirmationPolicy` exige dos lecturas consecutivas fuera de la zona antes de avisar. Se desactiva para zonas críticas.

```
public class ExitConfirmationPolicy {
    private final int required;           // lecturas consecutivas fuera, por
defecto 2
    private final double maxAccuracyMeters;

    public ExitConfirmationPolicy(int required, double maxAccuracyMeters) {
        this.required = required; this.maxAccuracyMeters = maxAccuracyMeters;
    }

    public boolean isUsable(Position p) { return p.accuracyMeters() <=
maxAccuracyMeters; }

    public boolean confirmed(List<Position> lastReadings, Geofence zone) {
        return lastReadings.size() >= required
            && lastReadings.stream().limit(required).allMatch(r ->
zone.isOutside(r.point()));
    }
}
```

`OutOfZoneRule` consulta esta política antes de crear el evento `ExitedZone`.

17. Del prototipo en Node al backend en Java

El prototipo (`server.js`) funcionó como prueba de concepto. Cada una de sus piezas tiene su reemplazo en el diseño definitivo:

Prototipo en Node	Backend en Java
Función `inside()` (algoritmo del rayo)	`PolygonGeofence.contains()`
Variable `animals` en memoria	`PositionRepository` y tablas `trackers` y `positions`
Archivo `zone.json`	Tabla `zones` y `ZoneService`
`POST /pos`	`POST /api/ingest/positions` y `PositionService.ingest()`
Transmisión SSE `/stream`	WebSocket `/ws/live` y `LivePositionPublisher`
Alerta cuando `was.out` cambia	`OutOfZoneRule` y `AlertDeduplicator`
Función `notify()` de Telegram	`TelegramNotifier`
Cola de reintentos del celular	Memoria sin conexión del GPS e idempotencia del servidor

Prototipo en Node	Backend en Java
Sin usuarios ni claves	`AuthService`, JWT y roles

Lo que el prototipo no tenía y el backend sí debe tener: base de datos, usuarios, varias fincas, historial, push con la app cerrada, vigilancia del silencio y pruebas automáticas.

18. Contrato de mensajes (JSON)

Estos formatos son el acuerdo entre servidor, GPS intermedio y app. Se definen como `record` en Java.

Entrada de una posición (`POST /api/ingest/positions`, con la cabecera `X-Ingest-Key`):

```
{
  "trackerCode": "TRK-0001",
  "lat": 4.8000,
  "lng": -74.0300,
  "recordedAt": "2026-10-19T14:20:00Z",
  "speedKmh": 1.2,
  "battery": 82,
  "accuracy": 8.5
}
```

```
public record PositionRequest(
    @NotBlank String trackerCode,
    @DecimalMin("-90") @DecimalMax("90") double lat,
    @DecimalMin("-180") @DecimalMax("180") double lng,
    @NotNull Instant recordedAt,
    Double speedKmh, Integer battery, Double accuracy) {}
```

Eventos por WebSocket (`/ws/live?farmId=`):

```
{ "type": "position", "animalId": "a-12", "lat": 4.8001, "lng": -74.0302,
  "status": "INSIDE", "battery": 82, "recordedAt": "2026-10-19T14:20:00Z" }

{ "type": "alert", "alertId": "al-98", "event": "EXITED_ZONE", "severity":
"HIGH",
  "animalName": "Vaca 12", "zoneName": "Potrero Norte", "metersOut": 140,
  "at": "2026-10-19T14:21:00Z" }
```

Mensaje push (FCM, solo datos), con el que la app decide el canal y el sonido:

```
{ "channel": "urgente", "title": "Vaca 12 salió de la zona",
  "body": "Potrero Norte · a 140 m del límite", "alertId": "al-98",
"animalId": "a-12" }
```

Formato único de error, para que la app muestre mensajes claros:

```
{ "code": "ZONE_NOT_FOUND", "message": "La zona no existe o no pertenece a su
finca" }
```

19. Configuración

Todo valor que cambia entre computadores o que sea secreto va fuera del código, en variables de entorno.

Variable	Uso
`HATO_DB_URL`, `HATO_DB_USER`, `HATO_DB_PASSWORD`	Conexión a PostgreSQL
`HATO_JWT_SECRET`	Firma de los tokens de sesión
`HATO_INGEST_KEY`	Clave que debe enviar Traccar al reenviar posiciones
`HATO_FCM_CREDENTIALS`	Ruta del archivo de credenciales de Firebase
`HATO_TG_TOKEN`, `HATO_TG_CHAT`	Telegram (opcional)
`HATO_GPS_TCP_PORT`	Puerto del receptor TCP (por defecto 5023)

```
spring:
  threads:
    virtual:
      enabled: true
  datasource:
    url: ${HATO_DB_URL}
    username: ${HATO_DB_USER}
    password: ${HATO_DB_PASSWORD}

hato:
  gps:
    tcp-port: ${HATO_GPS_TCP_PORT:5023}
  alerts:
    no-signal-factor: 3
    low-battery-percent: 20
    idle-hours: 6
    min-accuracy-meters: 50
    exit-confirmations: 2
```

```
@ConfigurationProperties(prefix = "hato.alerts")
public record AlertProperties(int noSignalFactor, int lowBatteryPercent, int
idleHours,
                              double minAccuracyMeters, int
exitConfirmations) {}
```

Así los umbrales de las reglas se ajustan sin recompilar, y `AlertProperties` se inyecta por constructor en cada regla.

20. Seguridad y operación

- **Acceso:** HTTPS en todo, tokens JWT de vida corta, verificación de rol y de pertenencia a la finca en cada ruta.
- **Entrada de GPS:** la ruta de ingesta solo acepta la clave de servicio. El puerto TCP debe aceptar únicamente códigos de GPS registrados y descartar el resto.
- **Datos:** claves de usuario con hash (BCrypt o Argon2), tokens y claves nunca en los registros (logs), copias de seguridad diarias y política de retención de posiciones.
- **Observabilidad:** `/actuator/health`, y métricas de posiciones recibidas por minuto, errores de decodificación y tiempo entre la llegada de una posición y el envío del aviso.
- **Vigilar al propio servidor:** una alarma externa si el backend deja de recibir posiciones de **todos** los GPS a la vez, que suele indicar un problema del servidor o de la red y no de los animales.

- **Hosting:** un servidor virtual pequeño (VPS) alcanza para empezar con decenas o cientos de GPS. Debe permitir tráfico TCP entrante si se usa receptor propio.

21. Criterios de aceptación del backend

El backend se considera listo para probar con el ganado cuando cumple esto:

1. Una posición fuera de la zona genera **una sola** alerta y un push, incluso si llegan varias posiciones seguidas.
2. La misma posición enviada dos veces no se duplica en el historial.
3. Una posición antigua que llega tarde no dispara alertas ni mueve el marcador a un lugar viejo.
4. Con la app cerrada, el push llega con el canal y el sonido correctos.
5. Un GPS sin reportar durante `noSignalFactor × intervalo` genera la alerta «sin señal».
6. Un usuario no puede ver animales, zonas ni alertas de otra finca.
7. El volver a la zona cierra la alerta y envía un aviso informativo.
8. Las pruebas automáticas de geometría, reglas y avisos pasan sin necesidad de levantar Spring.

Decisiones que siguen abiertas

- Traccar o receptor TCP propio como entrada definitiva.
- Si la finca necesita LoRa por falta de cobertura celular.
- Cuánto tiempo se conservan las posiciones (por ejemplo, 12 meses).
- Qué canales de respaldo se activan además del push (Telegram, SMS o correo).